

A Unified Framework for Recommendation, Try-On, and Try-Off in Virtual Fashion

Aaarat Chadda, Dheeraj , Taher Merchant

The intersection of fashion and artificial intelligence has witnessed explosive growth, driven by the need for personalized experiences and immersive product interaction in digital retail. This research introduces a tri-modular AI fashion framework combining graph-based recommendation, realistic virtual try-on, and innovative virtual try-off pipelines. Together, these systems represent a complete and intelligent virtual dressing room from discovering ideal garments to visualizing fit, style, and replacement possibilities.

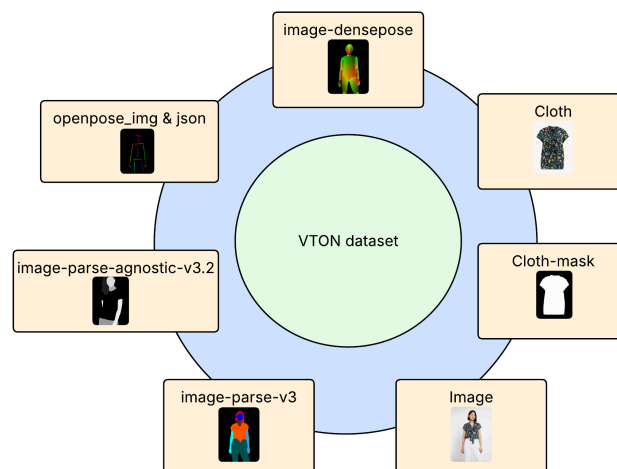
1. GAT-based Clothing Recommendation System: At the foundation of this framework lies a Graph Attention Network (GAT) that models the similarity between fashion items based on visual-language representations. Each clothing image is processed using a Visual Language Model to extract structured attributes such as color, fabric, sleeve length, and fit. These attributes are encoded into embeddings using transformer-based encoders (MiniLM), forming the node features of a fully connected clothing graph. Edges represent similarity across attribute space. The GAT is trained with triplet and contrastive losses, enabling robust clustering of similar items and yielding context-aware, fashion-aligned recommendations.

2. DCI-VTON: Dense Correspondence Inpainting Try-On: To enable realistic visualization of clothing on users, a Dense Correspondence Inpainting-based Virtual Try-On Network (DCI-VTON) is integrated. This GAN-based system estimates dense warping between the person image and the garment image, aligning the clothing to the target body while preserving natural appearance. A correspondence-aware generator performs spatial transformation, while an inpainting discriminator reconstructs occluded body parts and refines the composite for realism. The system excels in preserving garment texture and generating visually coherent try-on images across varied poses and body shapes.

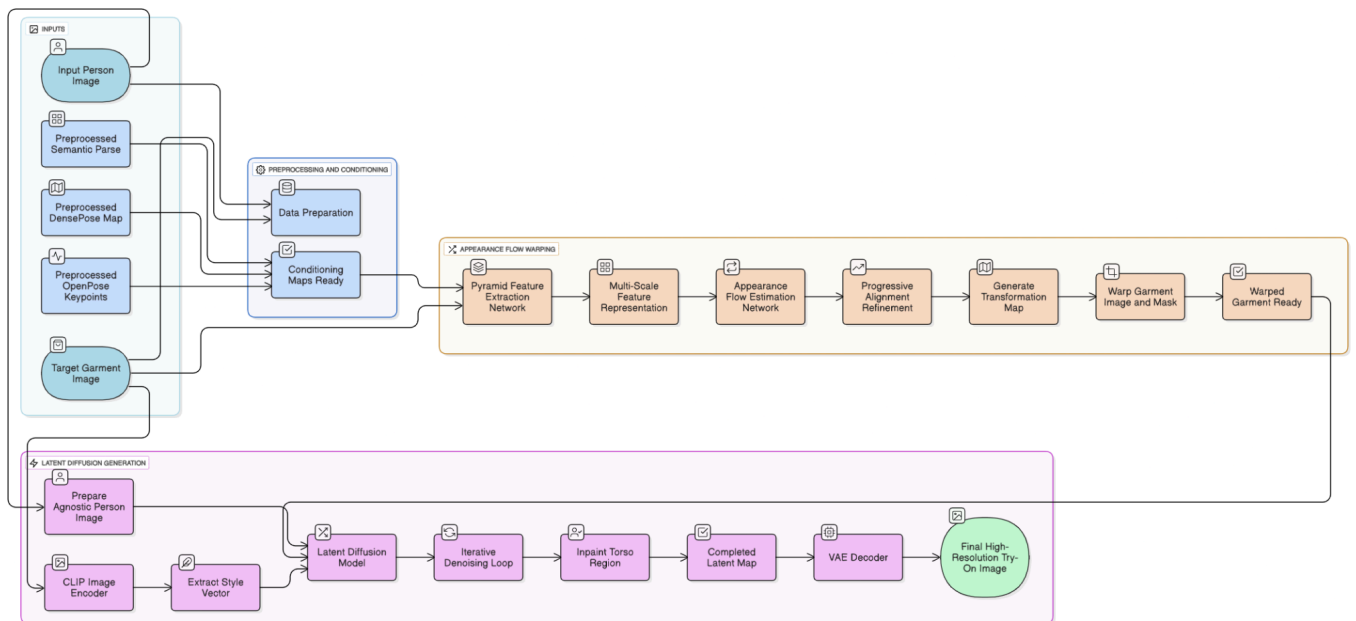
3. GAN-based Try-Off Module: A unique addition to this pipeline is the Try-Off GAN, designed to reverse the try-on process. This module removes the clothing from a worn-person image to reconstruct the underlying body or neutral base layer. It serves as a precursor to re-dressing the person in different garments or for clean pose transfer. The try-off GAN includes a specialized generator and discriminator trained to hallucinate plausible body features behind the clothing while preserving pose and context. This module enhances layering realism for sequential try-on applications and enables precise fitting for garments of different occlusion levels.

Virtual Tryon (DCI- VTON based)

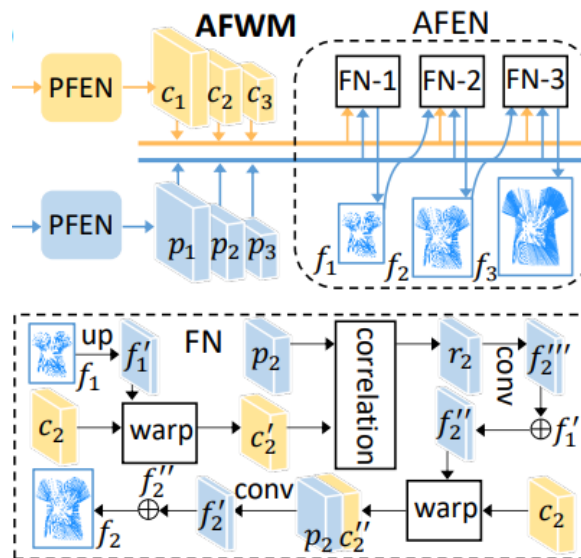
We began by carefully analyzing the dataset and conducting initial research. One key observation was that all images were high-resolution (768×1024), which opened up opportunities for exploring high-resolution VTON systems. Given our personal inclination toward Diffusion Models, we selected **DCI-VTON**, a two-stage pipeline, as the foundation for our work. A major limitation of the dataset was the **absence of DensePose annotations**, which are crucial for person-body alignment in VTON. To overcome this, we modified the **DensePose module from Detectron2**, a task that required substantial manual effort. We generated the complete set of **3-channel DensePose maps (I, U, V)** for each image in the dataset, enabling proper conditioning for the model. As a result, our final dataset included:



It has two stages -



i) AFMW (Appearance Flow Matching Module)



The Warping Module: Fitting the Clothes to the Person

The Warping Module is the core component responsible for realistically fitting the flat clothing image onto the target person's body. Its primary job is to intelligently reshape and position the

garment to match the person's pose, size, and body contours. It achieves this by generating a precise appearance flow.

What is Appearance Flow?

Think of the appearance flow as a detailed instruction map. For every pixel in the area of the person where the clothing should be, this map points to the exact corresponding pixel on the original, flat clothing image. By following this map, we can reconstruct the garment with the correct bends, folds, and stretches, creating a naturally warped clothing item.

How It Works: A Two-Step Process

The module uses a sophisticated deep learning architecture to calculate this flow, broken down into two main stages:

1. Feature Extraction (PFEN - Pyramid Feature Extraction Network)

First, the module needs to deeply understand both the clothing and the person. The PFEN analyzes both images at multiple levels of detail simultaneously like looking at them from far away and then zooming in closer and closer. This "pyramid" approach allows it to capture:

- High-level features: The overall shape and structure of the garment and the person's torso.
- Low-level features: Fine details like a logo, a specific pattern, or sharp stripes on the clothing.

2. Progressive Flow Estimation (AFEN - Appearance Flow Estimation Network)

Once it understands the features, the AFEN calculates the appearance flow. Instead of trying to get it perfect in one step, it builds the flow progressively, from coarse to fine, across the feature pyramid.

- Initial Guess: It starts with the low-resolution, blurry features to make a rough initial guess of the warp. This correctly aligns the overall shape.
- Iterative Refinement: It then moves to the next, more detailed feature level. Using the previous rough guess as a starting point, it calculates a small correction (a residual flow) by matching the finer details. This fixes misalignments and sharpens the fit.
- Final Result: This refinement process is repeated at each level until it reaches the highest resolution. The final result is a highly accurate flow map that can handle large

deformations while preserving critical clothing characteristics, ensuring logos and patterns remain intact and undistorted.

In essence, the Warping Module first creates a multi-layered understanding of the inputs and then progressively builds a precise transformation map, ensuring the final virtual try-on looks both natural and detailed

ii) Latent Diffusion Model (LDM).

Unlike a simple "copy-and-paste," this model understands human pose, body shape, and clothing physics to realistically render the garment on the person, complete with natural-looking drapes, folds, and lighting. The process is broken down into four main stages:

1. Data Preparation
2. Geometric Warping
3. High-Level Style Understanding with CLIP
4. Diffusion-Based Image Generation.

Stage 1: Data Preparation & Agnostic Representation

The model cannot work with raw images alone. It requires a rich set of intermediate representations to understand the scene. For a given person and cloth image, we first generate:

- **Human Pose Keypoints:** Using a model like OpenPose, we detect the 2D coordinates of the person's joints (shoulders, elbows, etc.). This provides the structural skeleton.
- **Semantic Segmentation Map:** Using a human parsing model, we create a map that labels every pixel of the person's image with its corresponding body part (e.g., hair, face, arms, legs, original clothing).
- **Agnostic Person Representation:** This is a crucial step. Using the pose keypoints and segmentation map, we create a "person-agnostic" image. In this image, the person's original clothing (specifically the upper body garment to be replaced), arms, and torso area are heavily blurred or masked out. What remains are the person's head, hair, and lower body.

Stage 2: Geometric Warping (The Prerequisite) - Provided by AFMW

- **Outputs:** The result is a Warped Cloth Image and a corresponding Warped Cloth Mask. This warped cloth is now geometrically consistent with the target person's posture.

Stage 3: High-Level Style Understanding with CLIP

While the warped cloth provides the local texture and pattern, the diffusion model also needs a global understanding of the clothing's overall style, color, and concept. This is where CLIP (Contrastive Language-Image Pre-Training) comes in.

- CLIP is a powerful model from OpenAI that was trained on hundreds of millions of image-text pairs from the internet. It learned to map both images and text descriptions into the same high-dimensional "embedding space." In this space, the vector for a photo of a "red silk shirt" is mathematically close to the vector for the text "a red silk shirt."
- We are using CLIP Image Encoder. We feed the original, clean, un-warped clothing image into the CLIP Image Encoder.
- CLIP processes the image and outputs a single, dense vector (e.g., a list of 768 numbers). This vector is a rich, numerical summary of the clothing item. It doesn't just capture the color red; it captures the concept of the item "a formal red silk shirt," "a casual red cotton t-shirt," or "a sporty red jersey." This vector acts as a high-level style guide for the diffusion model.
- By using CLIP, we provide a condition that tells the model not just what pattern to draw, but also what kind of item it is drawing. This helps ensure consistency in material properties, lighting, and overall aesthetic.

Stage 4: Diffusion-Based Inpainting & Generation

This is the core generative stage where the final image is created. It uses a Latent Diffusion Model for efficiency and high-quality results.

1. Encoding to Latent Space: Instead of working with large pixel-based images, we use a powerful Variational Autoencoder (VAE) to encode our inputs into a much smaller, compressed latent space. This includes:
 - The Agnostic Person Image from Stage 1.
 - The Warped Cloth Image from Stage 2.
2. The Inpainting Task: The model is framed as a conditional inpainting problem. It receives:
 - The Masked Input: The VAE-encoded agnostic person, which has a "hole" where the new clothing should be.
 - The Condition: A set of inputs that guide the generation process. This includes:

- The encoded Warped Cloth (tells the model what local texture to paint).
- The CLIP Style Vector from Stage 3
- The encoded agnostic person's features outside the mask (to ensure the new clothing blends seamlessly).

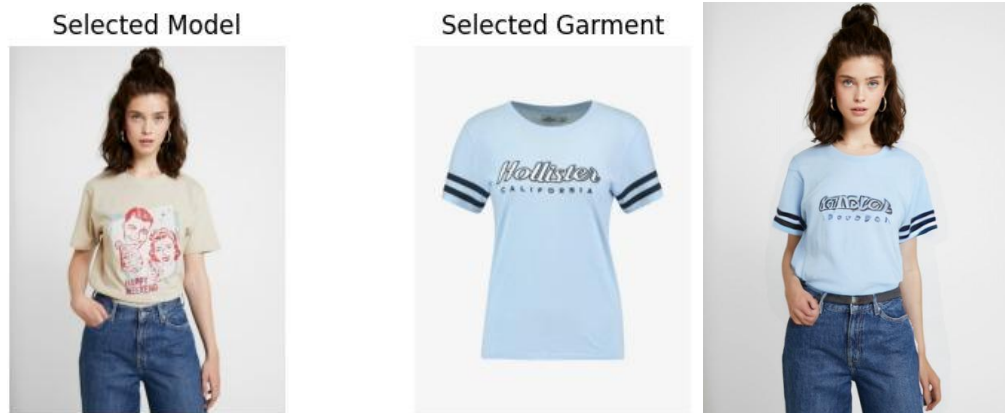
3. The Diffusion Process (Denoising):

- The model starts with pure random noise in the "hole" of the latent space.
- In a step-by-step iterative process (e.g., 50 steps), a powerful U-Net model predicts and removes a small amount of noise at each step.
- Crucially, at every step, the U-Net is guided by the condition using a mechanism called cross-attention. It constantly "looks at" the warped cloth texture and the CLIP style vector to ensure the noise it removes is replaced with details that are consistent with both the local pattern and the global style.

4. Decoding to Pixel Space: Once the denoising process is complete, we are left with a final, clean latent representation. The VAE's decoder is then used to convert this latent image back into a full-resolution pixel image.

5. Final Composition: The generated image of the new clothing is seamlessly composited onto the original agnostic person image, resulting in the final, photorealistic try-on output.

This multi-stage pipeline, combining geometric alignment with both local texture (warped cloth) and global style (CLIP) conditioning, allows the diffusion model to produce high-fidelity results that are robust to variations in human pose and clothing styles.



Results Of Virtual Try On Model

GAT-Powered Visual Search & Recommendation System

1. System Overview

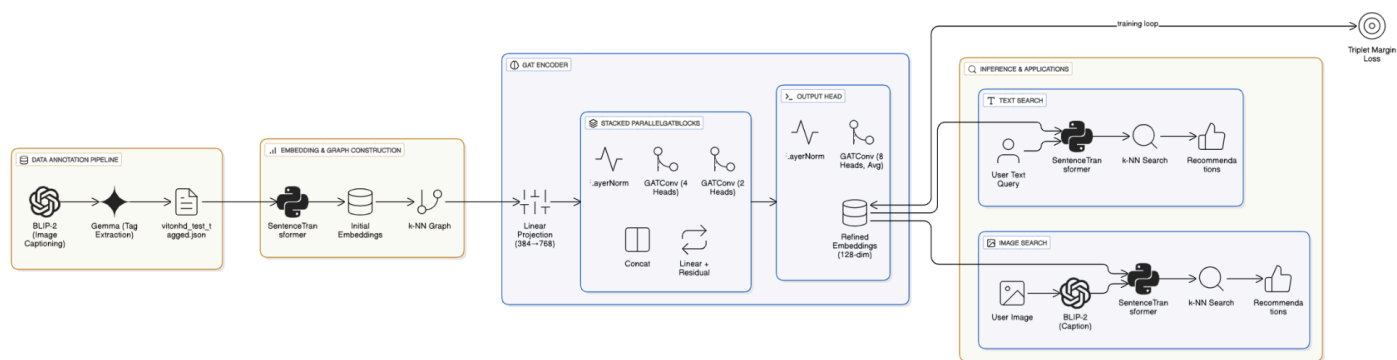
This document outlines the architecture and implementation of an advanced visual search and recommendation system for fashion e-commerce. The system is designed to provide highly relevant, context-aware recommendations for both text-based and image-based queries.

At its core, the system leverages a Graph Attention Network (GAT) to refine its understanding of the relationships between clothing items. Unlike traditional methods that treat each item in isolation, our GAT-based approach builds a comprehensive graph of the entire product catalog. This allows the model to learn nuanced similarities and contextual connections (e.g., "items that are often viewed together," "items with similar styles but different categories"), leading to a significantly more intuitive and accurate user experience.

The initial metadata for our clothing items was generated using a state-of-the-art multi-modal pipeline featuring Google's Gemma and BLIP-2, ensuring a rich, descriptive foundation for our graph.

2. System Architecture

The system is built on a multi-stage pipeline that transforms raw product images into a refined, searchable embedding space.



2.1. Data Annotation Pipeline (Pre-processing)

The foundation of any good recommendation system is high-quality data. We automated the creation of rich, descriptive tags for our entire clothing catalog using a powerful AI pipeline:

- Image Understanding (BLIP-2): Each product image was processed by the BLIP-2 model to generate a detailed textual description.
- Structured Tag Generation (Gemma): The descriptive text from BLIP-2 was then fed into a Gemma model, which was prompted to extract and structure this information into a consistent JSON format, identifying key attributes like tag_category (e.g., "Style", "Pattern", "Sleeve Length") and tag_name (e.g., "Casual", "Floral", "Long Sleeve").
- Output: The result of this process is the vitonhd_test_tagged.json file, which serves as the primary input for our model.

2.2. Initial Embedding and Graph Construction

Once we have the text data, we begin building the graph structure.

1. Text Embedding: The structured tags for each clothing item are concatenated into a single descriptive string. This string is then converted into a high-dimensional numerical vector (an embedding) using the SentenceTransformer ('all-MiniLM-L6-v2') model. This initial embedding captures the semantic meaning of the item's description.
2. Graph Construction (k-NN): To understand the relationships between items, we construct a similarity graph. Each clothing item becomes a node in the graph. We use the k-Nearest Neighbors (k-NN) algorithm to find the 5 most similar items for each item based on the cosine similarity of their initial embeddings. An edge is then created between an item and its neighbors, signifying an initial, strong similarity.

2.3. Graph Embedding Refinement (GAT)

This is the core innovation of the system. While the initial embeddings are good, they lack context. The GAT's role is to look at the entire graph and refine each item's embedding based on its neighbors' properties.

Our GATEncoder is a deep and complex architecture designed for maximum performance:

- Initial Projection: The input embeddings (384 dimensions) are first projected into a higher-dimensional space using a 1x1 convolution (a linear layer). This gives the model more capacity to learn complex features.

- **ParallelGATBlocks:** The network is composed of a series of stacked residual blocks. Each block processes the graph data through multiple parallel pathways:
- **Parallel Attention:** Two GATConv layers with different numbers of attention heads run in parallel. This allows the model to simultaneously capture different types of relationships between nodes.
- **1x1 Convolution:** The outputs of the parallel branches are concatenated and then unified using another 1x1 convolution, which projects the combined features back to the original dimension.
- **Residual Connection & Normalization:** Each block includes a skip connection and layer normalization, which are crucial for training a deep and stable graph network.
- **Final Output Head:** After passing through all the blocks, a final multi-head attention layer produces the refined, output embeddings (128 dimensions).

2.4. Training with Triplet Loss

The GAT model is trained using Triplet Margin Loss. The goal is to optimize the embedding space such that:

The distance between an anchor (a sample item) and a positive (a similar item) is minimized.

The distance between the anchor and a negative (a dissimilar item) is maximized.

We employ a semi-hard negative mining strategy to find the most informative training examples, which significantly speeds up convergence and prevents model collapse.

3. Inference and Use Cases

Once the GATEncoder is trained, it produces a final set of highly context-aware embeddings for every item in the catalog. These embeddings are saved and used for real-time similarity search.

3.1. Text-Based Search

1. A user enters a text query (e.g., "blue shirt with long sleeves").
2. The query is converted into an embedding using the same SentenceTransformer.
3. We perform a k-NN search (e.g., using cosine similarity) against the final GAT-refined catalog embeddings.
4. The top N most similar items are returned to the user.

3.2. Image-Based Search

1. A user uploads an image of a clothing item.

2. The image is passed through a model like BLIP-2 to generate a descriptive caption (e.g., "a blue long-sleeve shirt with a collar").
3. This caption is then treated as a text query and follows the same process as above, finding the most similar items in the GAT-refined embedding space.

This multi-modal approach provides a seamless and powerful visual search experience, powered by the contextual understanding of our Graph Attention Network.



SEMANTIC SEGMENTATION FOR GAT

GAN based Try - Off

Traditional Virtual Try-On (VTON) pipelines rely on clean, flat product images of garments typically captured in professional catalog settings. However, in many real-world applications (e.g., second-hand clothing resale, user-contributed garments, or unseen in-the-wild data), such catalog-style flat images are unavailable. To address this bottleneck, we developed a GAN-based Try-Off Module, a generative system that can isolate and transform garments directly from

person-worn images into clean, flat apparel representations. This module functions as a "Try-Off" system in contrast to VTON's "Try-On" by attempting to revert the worn clothing back to its flattened, standalone format, making it suitable for direct use in downstream try-on systems or for cataloging and retrieval.

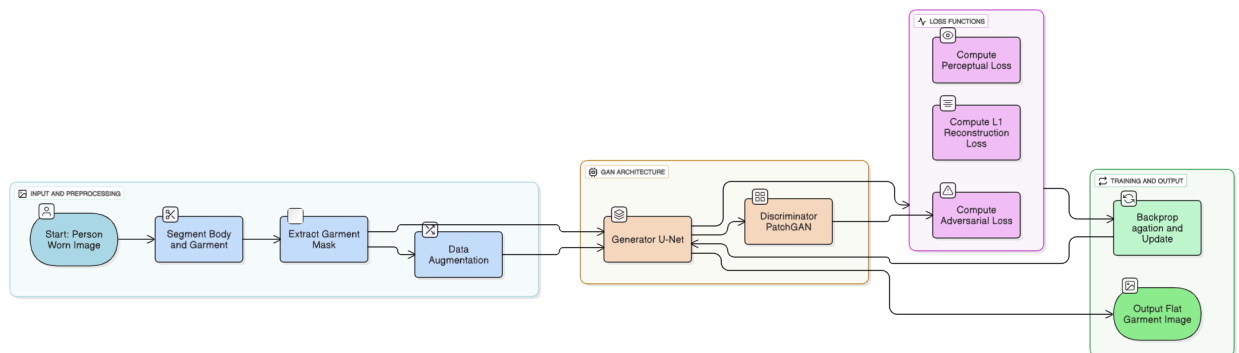
Architecture

Generator: ResNet34-Based U-Net: The generator uses an encoder-decoder U-Net architecture with skip connections to preserve spatial details while learning global context. The encoder is based on ResNet-34, pretrained on ImageNet for improved feature representation.

- Input: Concatenation of person image and optional cloth mask (from segmentation).
- Output: RGB flat image of the garment, matching resolution to the original.
- Skip connections: Enable recovery of local garment textures and borders.

Discriminator: PatchGAN: To ensure high-frequency detail preservation and realistic flat cloth generation, we use a 70×70 PatchGAN discriminator.

- Operates on local image patches instead of the entire image.
- Encourages texture-level realism and reduces blurring.
- Trained with Least Squares GAN (LSGAN) loss for stability.



Loss Functions

To ensure both realism and consistency, the following losses are used:

Adversarial Loss: Encourages the generator to produce realistic flat cloths that can fool the

discriminator.

$$\mathcal{L}_{GAN}(G, D) = \mathbb{E}[(D(I_{\text{flat}}))^2] + \mathbb{E}[(D(G(I_{\text{person}})) - 1)^2]$$

- L1 Reconstruction Loss: Penalizes pixel-wise differences between predicted and ground-truth flat images.

$$\mathcal{L}_{L1} = \|G(I_{\text{person}}) - I_{\text{flat}}\|_1$$

- Perceptual Loss: Measures distance in feature space using pretrained VGG16 layers to ensure semantic similarity.

$$\mathcal{L}_{\text{perc}} = \sum_i \|\phi_i(G(I)) - \phi_i(I_{\text{flat}})\|_2^2$$

Training Setup

- Dataset: Uses parsed clothing datasets with both worn-person images and corresponding flat garment images (e.g., VITON-HD, MPV).
- Input Preprocessing: Body regions are segmented using a pretrained human parser. Only upper-body garments are passed to the model.
- Augmentations: Random color jitter, flipping, and occlusion to simulate varied real-world conditions.
- Resolution: 256×192 or 512×384 depending on the dataset.



Output Samples

The model consistently produces flat garments that:

- Match the visual style and texture of the original clothing,
- Are devoid of body parts or background clutter,
- Can be directly used as inputs for VTON modules like DCI-VTON.

Conclusion

This work presents a unified, modular framework that brings together the core pillars of intelligent fashion technology: recommendation, virtual try-on, and virtual try-off. By integrating a Graph Attention Network (GAT)-based recommender system, a dense correspondence inpainting-based try-on pipeline (DCI-VTON), and a novel GAN-powered try-off generator, we deliver a complete end-to-end solution for virtual fashion interaction.

The GAT recommender learns nuanced, context-aware relationships between clothing items using structured visual-language features, enabling personalized and visually coherent suggestions. DCI-VTON sets a new standard for realism in virtual try-on, leveraging warping and diffusion models to produce high-resolution, photorealistic outputs that preserve garment textures and align with human pose. The try-off module complements this pipeline by reversing the try-on process extracting clean, flat garments from worn images which expands the system’s flexibility for re-dressing, styling, and fashion archival tasks.

Together, these components form a scalable and adaptable system that bridges the gap between AI and real-world digital fashion applications. Whether enhancing consumer experience in e-commerce, powering virtual closets, or assisting stylists and designers in garment prototyping, this unified framework represents a significant leap forward in AI-driven fashion intelligence. Future directions include integrating real-time interaction, expanding to full-body garments, and optimizing cross-modal compatibility for more personalized, immersive fashion experiences.